

CINEMAX

Manuale tecnico

INDICE

INSTALLAZIONE	3
Requisiti di sistema	3
Setup ambiente	3
Passo 1: Download del JDK	3
Passo 2: Installazione	3
Installazione programma	4
ESECUZIONE ED USO	6
MENU' INIZIALE	7
Primo Avvio & Menu Principale	7
1. Login	9
MENU CLIENTE	11
Crea prenotazione	12
Modifica prenotazione	15
Visualizza prenotazioni	18
Elimina prenotazione	19
Visualizza proiezioni	21
Logout	22
Chiudi programma	22
MENU PROIEZIONISTA	23
Aggiungi proiezione	23
Modifica proiezione	26
Elimina proiezione	29
Visualizza proiezioni	30
Logout	30
Chiudi programma	31
MENU BIGLIETTAIO	32
Visualizza prenotazioni di oggi	32
Cerca prenotazione	33
Logout	37
Chiudi programma	37
2. Registrarsi	38
3. Palinsesto	40
4. Esci	44
DATA SET	45
Logica di Singolarità delle Strutture Dati e Classi Statiche	45
Credenziali di Accesso per i Ruoli Utente	45
Proiezionisti	45
Bigliettai	46
Clienti	46

	Archivio Proiezioni e Storico Prenotazioni	46
LIMITI		48

INSTALLAZIONE

Requisiti di sistema

- CPU con architettura a 64 bit;
- OS: Windows 11 / macOS

I requisiti sono consultabili con maggior dettaglio direttamente sul sito di Oracle presso: [Oracle Java SE Universal Subscription Documentation](#)

Setup ambiente

Per poter eseguire CINEMAX, è necessario installare sul computer il Java Development Kit (JDK) 25 o superiore. Se si ha già un JDK installato, verificare la versione aprendo il terminale e digitando:

java -version

Passo 1: Download del JDK

1. Collegarsi al sito ufficiale per scaricare il JDK: [Java Downloads | Oracle Italia](#)
2. Selezionare la versione **JDK 25 o superiore**.
3. Scegliere il pacchetto adatto al proprio sistema operativo:
 - **Windows:** scaricare l'installer **.exe** (x64).
 - **macOS:** scaricare l'installer **.dmg** (scegliere *Architecture: aarch64* se si ha un Mac con chip M1/M2/M3, altrimenti x64).
 - **Linux(Ubuntu/Debian):** scaricare il pacchetto **.deb** adatto alla propria architettura o usare il gestore pacchetti della propria distribuzione.

Passo 2: Installazione

- **Su Windows/macOS:** aprire il file scaricato e seguire la procedura guidata cliccando sempre su "Avanti" (Next) fino al completamento. Non modificare la cartella di installazione predefinita.
- **Su Linux (Ubuntu/Debian):** fare **doppio clic** sul file **.deb** scaricato. Si aprirà il software center della propria distribuzione: cliccare su **Installa**, inserire la password di sistema e attendere il completamento.

Metodo da Terminale (Consigliato se il metodo grafico si blocca): aprire la cartella dove è scaricato il file (es. *Scaricati* o *Downloads*). Cliccare con il **tasto destro** in un punto vuoto della cartella e selezionare **"Apri nel terminale"** (oppure *"Apri terminale qui"*). Digitare il seguente comando (sostituisci **nome_file.deb** con il nome esatto del file scaricato, aiutandosi premendo il tasto **TAB** per l'autocompletamento):

sudo dpkg -i nome_file.deb

Inserire la propria password e premere Invio. Se alla fine dell'installazione il terminale segnala errori di dipendenze mancanti, si possono risolvere subito lanciando:

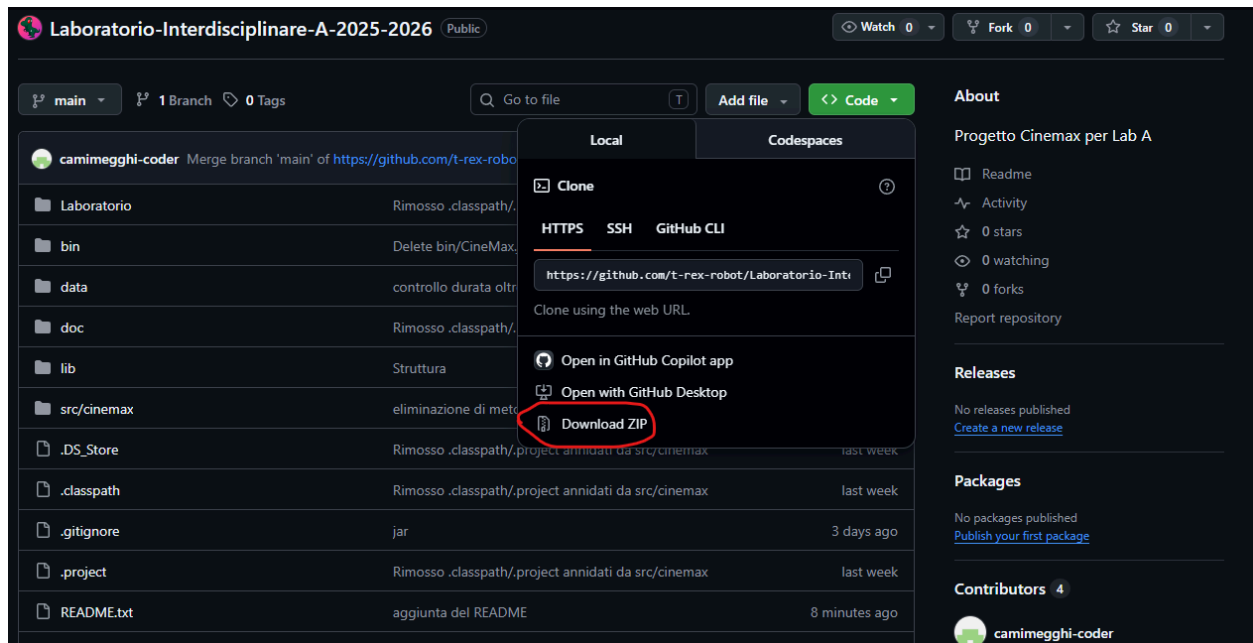
```
sudo apt install -f
```

Installazione programma

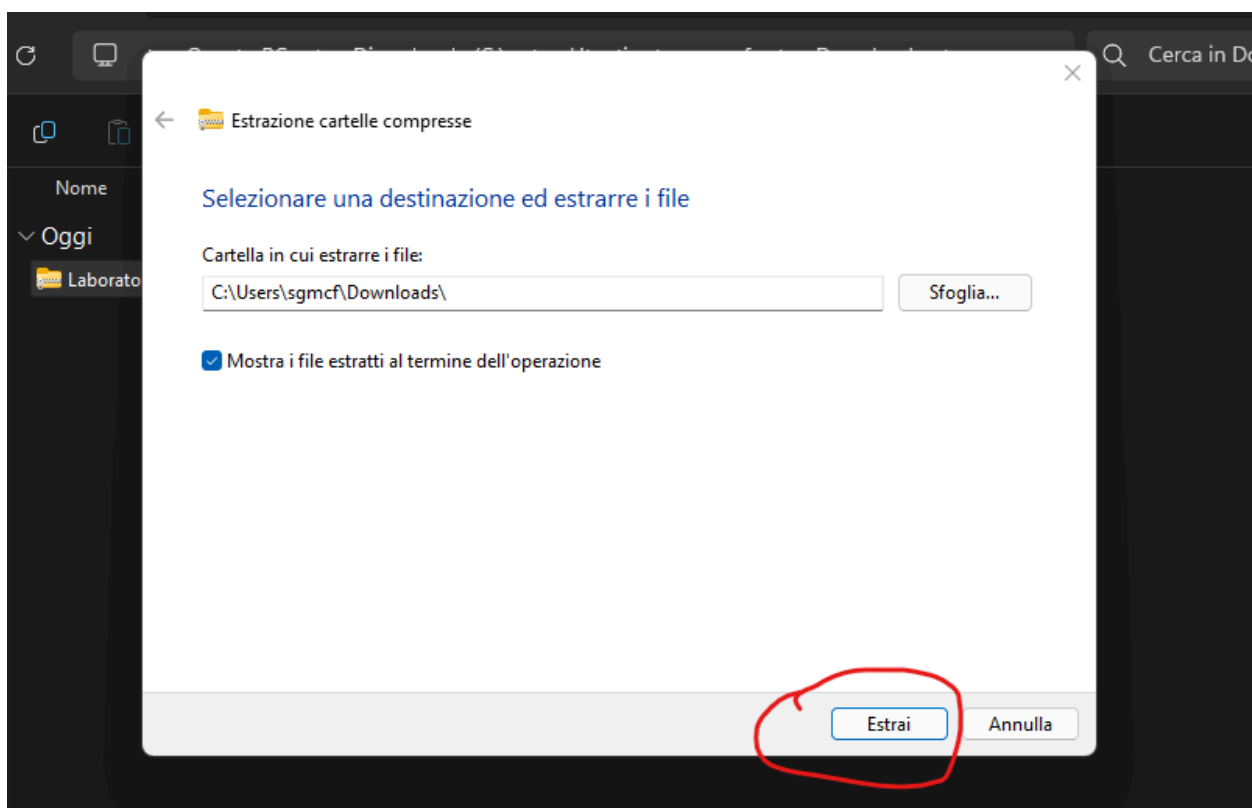
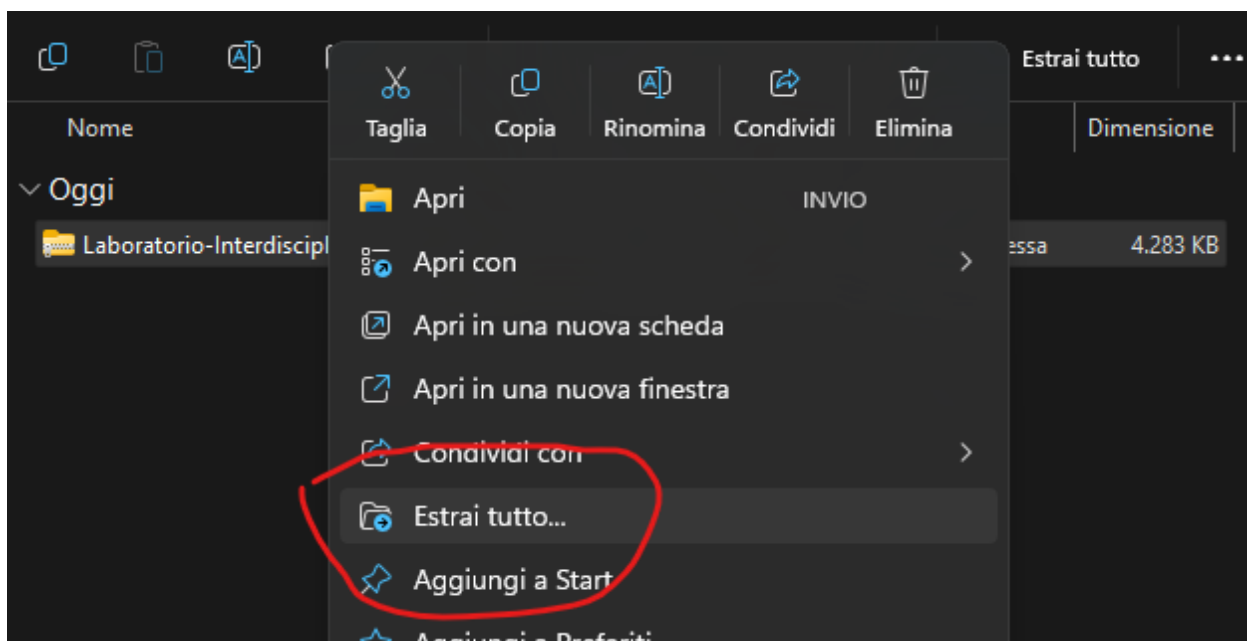
Trattandosi di un'applicazione eseguibile da interfaccia a riga di comando, il software non richiede procedure di installazione automatizzate nel sistema operativo, ma si basa sul posizionamento e sull'estrazione dei file di progetto.

La procedura di installazione e configurazione iniziale si articola nei seguenti passaggi:

1. **Download:** accedere al repository remoto tramite il link ufficiale di GitHub del progetto (<https://github.com/t-rex-robot/Laboratorio-Interdisciplinare-A-2025-2026.git>) e procedere al download del pacchetto dell'applicazione, rilasciato sotto forma di file ZIP.



2. **Estrazione del pacchetto:** localizzare l'archivio scaricato sul proprio computer ed estrarlo. L'operazione genererà la cartella [Laboratorio-Interdisciplinare-A-2025-2026](#).



ESECUZIONE ED USO

Per lanciare l'applicazione, il terminale deve essere posizionato all'interno della cartella principale del programma, in modo da consentire a Java di rintracciare i file di configurazione. Si può orientare il terminale seguendo una di queste due modalità:

- **Modalità 1:** aprire la cartella principale **Laboratorio interdisciplinare 2025-2026**, fare click con il tasto destro del mouse in un punto vuoto e selezionare la voce **"Apri nel terminale"** (o "Apri terminale qui" / "Apri in Terminale Windows" a seconda del sistema operativo).
- **Modalità 2:** aprire la cartella principale **Laboratorio interdisciplinare 2025-2026** e copiare il percorso dalla barra degli indirizzi in alto. Successivamente, aprire il terminale, digitare il comando **cd** seguito da uno spazio e incollare il percorso copiato (ad esempio **cd C:\Utenti\Desktop\Laboratorio-Interdisciplinare-A-2025-2026**) e premere **Invio**.

A questo punto si può avviare l'eseguibile posizionato nella cartella **bin** con uno dei seguenti metodi:

1. **Digitazione con autocompletamento:** digitare il comando base puntando alla sotto-cartella:

java -jar bin/C
2. A questo punto, è sufficiente premere il tasto **TAB** sulla tastiera: il terminale completerà automaticamente il nome del file in **java -jar bin/Cinemax.jar**.
3. **Trascinamento del file:** digitare nel terminale il comando **java -jar** seguito da uno spazio. Aprire separatamente la cartella **bin**, selezionare il file **Cinemax.jar** con il mouse e **trascinarlo direttamente all'interno della finestra del terminale**.
4. **Copia e incolla del file:** digitare nel terminale **java -jar** seguito da uno spazio. Andare nella cartella **bin**, fare click con il tasto destro sul file **Cinemax.jar** e selezionare "Copia". Tornare sul terminale e incollare.

Infine, premere il tasto **Invio** della tastiera per eseguire il comando e visualizzare a schermo il Menu Iniziale.

MENU' INIZIALE

Primo Avvio & Menu Principale

Appena avviato il programma, ci si trova nel menu principale come **Utente Guest**.



Da qui ci si muove usando semplicemente la tastiera: digitando il numero dell'azione che si vuole fare e premendo **Invio**.

- **1. Login:** entra nel profilo dell'Utente (se già registrato);
- **2. Registrati:** crea un nuovo account;
- **3. Palinsesto:** sfoglia tutti i film disponibili al cinema;
- **4. Esci:** si chiude il programma.

All'avvio dell'applicazione, il sistema provvede a riempire le strutture dati interne caricando le informazioni persistenti presenti nei file d'archivio (palinsesto, account utente e prenotazioni esistenti).

Come strutture dati principali per la gestione delle informazioni in memoria sono state scelte le **HashMap**, al fine di garantire la massima efficienza dell'applicazione. La **HashMap**, infatti, offre una complessità temporale di accesso costante $O(1)$ per le operazioni fondamentali di inserimento, ricerca e rimozione degli elementi.

L'inizializzazione della memoria avviene in modo differenziato tramite metodi dedicati per ciascuna entità (Utenti, Film e Proiezioni). Come esempio viene mostrata l'implementazione del metodo **caricaUtentiDaFile()**, responsabile del caricamento dei profili d'accesso.

```
public static void caricaUtentiDaFile() throws FormatoDatiNonValidoException {
    try (BufferedReader br = new BufferedReader(new FileReader(PATH))) {
        String riga;
        while ((riga = br.readLine()) != null) {
            String[] dati = riga.split(",");
            if (dati.length != 7) throw new FormatoDatiNonValidoException ("Errore nel formato dei dati: attesi 7 campi, trovati " + dati.length + " campi.");
            Utente u = new Utente(dati[0], dati[1], dati[2], dati[3], LocalDate.parse(dati[4]), dati[5], Ruolo.valueOf(dati[6]));
            mappaUtenti.put(u.getUsername(), u);
        }
    } catch (IOException e) {
        System.err.println("Errore caricamento: " + e.getMessage());
    }
}
```

Il metodo opera secondo la seguente logica lineare:

- **Istanziamento dell'input stream:** viene inizializzato un oggetto **BufferedReader** orientato verso il percorso del file CSV di riferimento tramite la costante **PATH**.
- **Parsing e Validazione:** il flusso scorre il file riga per riga, separando i campi tramite il metodo **.split(",")**. Il sistema verifica la conformità del record controllando che il numero di campi sia esattamente pari a quello atteso (7 campi nel caso degli utenti). Qualora si riscontrasse un'anomalia nel tracciato record, il programma interrompe il processo lanciando un'eccezione personalizzata di tipo **FormatoDatiNonValidoException**.
- **Istanziamento e Allocazione:** in caso di esito positivo della validazione, i dati letti (tra cui la data di nascita, convertita mediante **LocalDate.parse()**, e l'enumerativo **Ruolo**) vengono utilizzati per istanziare un nuovo oggetto di tipo **Utente**. L'oggetto viene infine inserito all'interno della mappa globale **mappaUtenti**, utilizzando lo username come chiave di indicizzazione (poiché non possono esistere utenti con lo stesso username).

Il metodo è il medesimo per i Film e le Prenotazioni.

L'**HashMap** **mappaFilm** usa come chiave una stringa che concatena la data e l'ora (poiché non possono esistere due film con gli stessi valori temporali, in quanto il cinema è monosala)

L'**HashMap** **mappaPrenotazioni** usa come chiave il codice univoco che identifica ogni prenotazione.

1. Login

Richiede di inserire il proprio **username** e la propria **password**.

Il metodo `loginHashed` confronta la password inserita dall'utente con l'hash SHA-256 invocando il metodo ausiliario `passwordMatches`

```
public static boolean loginHashed(String username, String p) {
    if (mappaUtenti.containsKey(username)) {
        try {
            if (passwordMatches(username, p)) {
                System.out.println("login effettuato correttamente!");
                utenteCorrente = mappaUtenti.get(username);
                return true;
            }
        } catch (NoSuchAlgorithmException e) {
            System.err.println("Errore hashing password: " + e.getMessage());
            return false;
        }
        System.out.println("Username o Password errati!");
        return false;
    }
    System.out.println("Utente non registrato!");
    return false;
}
```

```
public static boolean passwordMatches(String username, String plainPassword) throws NoSuchAlgorithmException {
    if (!mappaUtenti.containsKey(username)) {
        return false;
    }
    String storedPassword = mappaUtenti.get(username).getPassword();
    String hashedInput = PasswordHasher.hashPassword(plainPassword);
    return storedPassword.equals(hashedInput) || storedPassword.equals(plainPassword);
}
```

Il processo di identificazione e profilazione dell'utenza è gestito dal metodo statico `loginHashed()`, implementato per convalidare l'identità dell'utente e reindirizzare il flusso di controllo verso l'interfaccia operativa di competenza.

Prima di eseguire qualsiasi computazione crittografica, il metodo interroga la struttura dati in memoria invocando la condizione `mappaUtenti.containsKey(username)`.

Se l'identificativo non viene riscontrato all'interno della mappa, il flusso si interrompe immediatamente stampando a terminale l'avviso `Utente non registrato`. Questo controllo preventivo evita calcoli di hashing superflui su stringhe non presenti in archivio.

Qualora lo username esista, il sistema delega la verifica delle credenziali al metodo ausiliario `passwordMatches()`. Questo metodo estrae la stringa memorizzata nel database locale (`storedPassword`) e implementa una logica flessibile basata su un operatore booleano `OR (||)`.

La password immessa in chiaro dall'utente viene convertita tramite il metodo `hashPassword()`. Il digest esadecimale calcolato a runtime (`hashedInput`) viene confrontato con la stringa presente nel record dell'archivio.

Se il primo confronto fallisce, il sistema verifica se la stringa memorizzata corrisponda direttamente alla password in chiaro (`plainPassword`). Questo passaggio logico assicura la

piena compatibilità con gli account di test pre-configurati inseriti in chiaro nei file di testo d'archivio.

Se le credenziali sono corrette, il sistema sbloccherà le funzionalità avanzate e mostrerà in automatico il **Menu Personalizzato** in base al ruolo dell'utente, ottenuto tramite metodo `getUtenteCorrente()`. Il menu personalizzato viene scelto tramite il metodo `menuRuolo()` che, tramite uno *switch-case* che analizza il campo enumerativo `Ruolo` dell'utente corrente, invoca il menu corrispondente.

```
boolean successo =
    GestioneUtenti.LoginHashed(username, password);

if (successo) {

    utenteLoggato =
        GestioneUtenti.getUtenteCorrente();

    System.out.println(
        "Benvenuto " +
        utenteLoggato.getUsername()
    );

    // entra subito nel menu corretto
    menuRuolo();
}
```

```
private void menuRuolo() {

    switch (utenteLoggato.getRuolo()) {

        case CLIENTE -> menuCliente();

        case BIGLIETTAIO -> menuBigliettaio();

        case PROIEZIONISTA -> menuProiezionista();

        default -> {
            System.out.println("Ruolo non valido");
            utenteLoggato = null;
        }

    }

}
```

La clausola `default` intercetta eventuali anomalie o ruoli non conosciuti.

MENU CLIENTE

```
--- MENU CLIENTE ---  
1. Crea prenotazione  
2. Modifica prenotazione  
3. Visualizza prenotazioni  
4. Elimina prenotazione  
5. Visualizza proiezioni  
6. Logout  
7. Chiudi programma  
  
Scelta:
```

Dopo aver effettuato l'autenticazione come Cliente, il programma mostra il menu principale dedicato alle funzionalità utente. La navigazione segue la logica standard dell'applicazione: è sufficiente digitare il numero corrispondente all'operazione desiderata in corrispondenza della voce scelta e premere il tasto **Invio**.

All'interno di questa interfaccia sono disponibili le seguenti opzioni:

1. **Crea prenotazione:** consente di avviare la procedura per riservare i biglietti, selezionando il film desiderato e i posti in sala.
2. **Modifica prenotazione:** permette di variare i dettagli di una prenotazione esistente.
3. **Visualizza prenotazioni:** mostra le prenotazioni effettuate dall'account corrente.
4. **Elimina prenotazione:** consente di cancellare una prenotazione attiva.
5. **Visualizza proiezioni:** mostra il palinsesto completo dei film attualmente in programmazione, con i relativi orari e sale.
6. **Logout:** disconnette l'account corrente e riporta l'utente al menu iniziale (Guest).
7. **Chiudi programma:** termina immediatamente l'esecuzione dell'applicazione.

Crea prenotazione

```
Scelta: 1
=== PRENOTAZIONE BIGLIETTI ===
Inserisci titolo film:
A Beautiful Mind
Inserisci data (yyyy-MM-dd):
2027-12-30
Inserisci ora (HH:mm):
10:30
Inserisci numero biglietti:
6
Prenotazione creata!
Prenotazione effettuata con successo!

Premere INVIO per continuare...
```

Per garantire il corretto funzionamento del programma ed evitare errori di validazione, è necessario rispettare i seguenti criteri di input:

- **Titolo film:** inserire il nome (anche parziale) dell'opera cinematografica desiderata.
- **Data della proiezione:** specificare la data seguendo tassativamente il formato standard richiesto **YYYY-MM-DD** (Anno-Mese-Giorno, es. **2027-12-30**).
- **Ora della proiezione:** inserire l'orario nel formato a 24 ore **HH:mm** (Ora:Minuti, es. **10:30**).
- **Numero biglietti:** digitare la quantità di posti che si desidera riservare.

Nota sulla capienza: la capacità massima di ciascuna sala è rigidamente fissata a **200 posti**. Il sistema verificherà in tempo reale la disponibilità residua e rifiuterà la richiesta qualora il numero di biglietti desiderati superi i posti ancora liberi.

Al termine dell'inserimento, se tutti i dati sono validi, il terminale mostrerà il messaggio **Prenotazione effettuata con successo!**. Sarà quindi sufficiente premere il tasto **Invio** per confermare e tornare al Menu Cliente.

Controlli in Crea Prenotazioni

La funzione di creazione della prenotazione è regolata da una sequenza di controlli di validazione finalizzati a garantire la consistenza dei dati e la corretta gestione della disponibilità dei posti in sala. Solo qualora tutte le verifiche abbiano esito positivo il sistema procede alla registrazione definitiva della prenotazione.

- **Controlli di Validità dell'Input:** le prime verifiche riguardano la correttezza dei dati inseriti e la validità temporale della proiezione.

```

// controllo numero valido
if (numeroBiglietti <= 0) {
    return false;
}

// controllo data
LocalDate oggi = LocalDate.now();

if (film.getData().isBefore(oggi)) {
    System.out.println("Non puoi prenotare per una data passata!");
    return false;
}else {
    if(film.getData().equals(oggi) && film.getOra().isBefore(LocalTime.now())) {
        System.out.println("La proiezione è già avvenuta o è in corso!");
        return false;
    }
}
}

```

Il sistema verifica che il numero di biglietti richiesto sia strettamente maggiore di zero. Qualora venga inserito un valore nullo o negativo, la procedura viene immediatamente interrotta restituendo esito negativo.

Attraverso il confronto con `LocalDate.now()`, il software impedisce la prenotazione di proiezioni già concluse. Se la data della proiezione risulta antecedente a quella corrente, l'operazione viene annullata. Mentre nel caso in cui la proiezione sia programmata nella giornata corrente, viene eseguito un ulteriore controllo mediante `LocalTime.now()`. Se l'orario di inizio risulta precedente all'istante corrente, il sistema considera la proiezione già iniziata o terminata e impedisce la creazione della prenotazione.

- **Controllo della Disponibilità dei Posti:** una volta superati i controlli di input, il sistema verifica la disponibilità residua della sala.

```

// controllo posti disponibili
if (film.getPostiSala() < numeroBiglietti) {

    System.out.println("Posti insufficienti!");
    return false;
}

```

Il numero di posti ancora disponibili viene confrontato con il numero di biglietti richiesti mediante il metodo `film.getPostiSala()`. Se la disponibilità risulta insufficiente, l'operazione viene annullata e viene notificata l'impossibilità di completare la prenotazione. Questo controllo impedisce il fenomeno dell'overbooking e garantisce che il numero di posti occupati non superi mai la capienza della sala.

Una volta superati i controlli il sistema svolge altre due importanti operazioni prima di concludere il processo:

- **Generazione del Codice Identificativo:** superati tutti i controlli di validazione, il sistema genera automaticamente un codice identificativo univoco per la prenotazione.

```
// Genera un codice univoco per la prenotazione usando UUID  
  
private static String generaCodicePrenotazione() {  
    return UUID.randomUUID().toString();  
}
```

L'identificativo viene creato mediante la classe `UUID` di Java (`UUID.randomUUID()`), garantendo l'unicità del codice senza la necessità di mantenere contatori incrementali o identificativi sequenziali.

- **Aggiornamento della Disponibilità:** prima della registrazione definitiva della prenotazione, il sistema aggiorna lo stato della proiezione riducendo il numero di posti ancora disponibili.

```
// aggiorna posti film  
film.eliminaPosti(numeroBiglietti);
```

Persistenza dei Dati (Caso di Successo)

Quando tutte le verifiche vengono superate, il sistema procede alla registrazione permanente della prenotazione.

```
public static void salvaPrenotazioniSuFile() {  
    //Uso filewriter(PATH) perche riscrive tutto per evitare duplicati e tenere lo stato aggiornato  
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(PATH))) {  
        // scorre tutte le prenotazioni  
        for (Prenotazione p : mappaPrenotazioni.values()) {  
            bw.write(p.toCSV());  
            bw.newLine();  
        }  
    } catch (IOException e) {  
        System.err.println(  
            "Errore salvataggio prenotazioni: " |  
            + e.getMessage()  
        );  
    }  
}
```

- **Allocazione in memoria:** viene istanziato un nuovo oggetto di tipo `Prenotazione` popolato con i parametri validati e allocato all'interno della struttura dati centralizzata `mappaPrenotazioni`.
- **Scrittura su File CSV:** il software apre un flusso di output stream gestito tramite un blocco `try-with-resources` che inizializza un `BufferedWriter` configurato in modalità `append` (`new FileWriter(PATH, true)`).
- **Serializzazione personalizzata:** l'oggetto viene serializzato in formato stringa CSV invocando il metodo `toCSV()`, il quale concatena i campi dell'oggetto intervallandoli con il delimitatore virgola (,). Il flusso provvede infine a scrivere la stringa sul file sul disco e ad aggiungere un terminatore di riga (`newLine()`).

Modifica prenotazione

```
Scelta: 2
Inserisci codice prenotazione:
77ee60ce-fec0-41d7-8403-c0494da5a263
Nuovo numero biglietti:
4
Prenotazione modificata con successo
Prenotazione modificata con successo!

Premere INVIO per continuare...
```

Questa opzione consente di variare esclusivamente il numero di biglietti associati a un determinato ordine, senza dover ripetere l'intera procedura di selezione del film.

La procedura richiede l'inserimento dei seguenti dati:

- **Codice prenotazione:** è necessario inserire (o incollare) l'identificativo alfanumerico univoco della prenotazione che si intende modificare, recuperabile tramite la funzione di visualizzazione delle prenotazioni.
- **Nuovo numero biglietti:** consente di specificare la nuova quantità totale di posti da riservare per la proiezione scelta.

Nota sui vincoli: anche in fase di modifica, il sistema verificherà che la nuova quantità non superi il limite di capienza della sala (pari a 200 posti complessivi) e aggiornerà automaticamente il costo totale della transazione in base al nuovo valore inserito.

Una volta confermato l'input, il sistema mostrerà il messaggio di conferma **Prenotazione modificata con successo**. Sarà quindi sufficiente premere il tasto **Invio** per completare l'operazione e ritornare alla schermata del Menu Cliente.

Controlli in Modifica Prenotazioni

La funzione di modifica è regolata da una sequenza di controlli finalizzati a preservare la coerenza delle prenotazioni e la disponibilità dei posti della sala.

- **Controlli di Esistenza e Validità Temporale:** le prime verifiche riguardano la validità della prenotazione e la possibilità di modificarla.

```
Prenotazione p = mappaPrenotazioni.get(codice);

if (p == null) {
    System.out.println("Prenotazione non trovata");
    return false;
}
```

Il sistema ricerca la prenotazione all'interno della struttura dati **mappaPrenotazioni** utilizzando il codice identificativo fornito. Se il codice non corrisponde ad alcuna prenotazione registrata, la procedura viene interrotta e viene notificato il messaggio "Prenotazione non trovata".

```
// controllo data (solo futuro)
if (p.getDataProiezione().isBefore(LocalDate.now()) || (p.getDataProiezione().equals(LocalDate.now()) && p.getOraProiezione().isBefore(LocalTime.now()))) {
    System.out.println("Impossibile modificare una proiezione passata");
    return false;
}
```

Attraverso il confronto con **LocalDate.now()** e **LocalTime.now()**, il sistema impedisce la modifica di prenotazioni riferite a proiezioni già svolte o già iniziate.

Procedimento di Modifica

L'aspetto principale della logica di business consiste nel ricalcolo dei posti disponibili in funzione della variazione del numero di biglietti. L'algoritmo confronta il numero di biglietti originariamente prenotati con il nuovo valore richiesto e distingue due scenari differenti.

```
// CASO 1: aumento biglietti
if (nuoviBiglietti > vecchiBiglietti) {

    int differenza = nuoviBiglietti - vecchiBiglietti;

    if (film.getPostiSala() < differenza) {
        System.out.println("Posti insufficienti");
        return false;
    }

    film.eliminaPosti(differenza);
}
```

Se il nuovo numero di biglietti risulta superiore a quello precedente, il sistema calcola la differenza tra i due valori. Successivamente verifica che il numero di posti ancora disponibili nella sala sia sufficiente a soddisfare la richiesta. In caso contrario la modifica viene annullata per evitare situazioni di overbooking. Se il controllo ha esito positivo, i posti aggiuntivi vengono sottratti dalla disponibilità della sala mediante il metodo `film.eliminaPosti()`.

```
// CASO 2: riduzione biglietti
else if (nuoviBiglietti < vecchiBiglietti) {

    int differenza = vecchiBiglietti - nuoviBiglietti;
    film.aggiungiPosti(differenza);
}
```

Se il nuovo numero di biglietti è inferiore a quello originario, il sistema determina il numero di posti da liberare e provvede a reinserirli nella disponibilità della sala tramite il metodo `film.aggiungiPosti()`. In questo modo il numero di posti disponibili rimane sempre coerente con il numero effettivo di biglietti prenotati.

Terminato l'aggiornamento della disponibilità della sala, il sistema modifica il numero di biglietti memorizzato all'interno dell'oggetto `Prenotazione` mediante il metodo `setNumeroBiglietti()`. La prenotazione mantiene invariati tutti gli altri attributi, compresi il codice identificativo, l'utente associato, il titolo della proiezione e la data dell'evento.

```
// aggiorna prenotazione
p.setNumeroBiglietti(nuoviBiglietti);

GestioneFilm.salvaFilmSuFile();

salvaPrenotazioniSuFile();
```

Visualizza prenotazioni

```
Scelta: 3
===== PRENOTAZIONE =====
Codice: 77ee60ce-fec0-41d7-8403-c0494da5a263
Cliente: leog
Data: 2027-12-30
Ora: 10:30
Biglietti: 6
Costo unitario: 12.0
Totale: 72.0
```

Selezionando l'opzione **3** dal Menu Cliente, il programma mostra i dettagli delle prenotazioni associate all'account in uso. Per ciascuna transazione vengono stampate a schermo le seguenti informazioni:

- **Codice:** un identificativo univoco alfanumerico generato automaticamente dal sistema. Questo codice è indispensabile per identificare la prenotazione qualora si desideri modificarla o eliminarla in seguito.
- **Cliente:** lo username dell'account che ha effettuato l'operazione.
- **Data e Ora:** i dettagli temporali relativi alla proiezione cinematografica selezionata.
- **Biglietti:** il numero di posti riservati per quella specifica sessione.
- **Costo unitario e Totale:** il prezzo del singolo titolo d'ingresso e il calcolo complessivo dell'importo dovuto, calcolato in base al numero di biglietti prenotati.

Consiglio di utilizzo: data la complessità e la lunghezza del codice univoco della prenotazione, si raccomanda di non digitarlo manualmente. È possibile selezionarlo direttamente dal terminale, copiarlo e incollarlo quando richiesto dalle funzioni di modifica o annullamento.

Estrazione delle Informazioni

Il metodo accede ai campi dell'oggetto **Prenotazione** mediante i relativi metodi getter. Le informazioni vengono quindi stampate in formato testuale attraverso una sequenza ordinata di istruzioni `System.out.println()`, precedute dall'intestazione "==== PRENOTAZIONE ====", al fine di migliorarne la leggibilità.

```
public static void visualizzaPrenotazione(Prenotazione p) {  
  
    System.out.println("==== PRENOTAZIONE ====");  
    System.out.println("Codice: " + p.getCodicePrenotazione());  
    System.out.println("Cliente: " + p.getUsername());  
    System.out.println("Titolo: " + p.getTitoloFilm());  
    System.out.println("Data: " + p.getDataProiezione());  
    System.out.println("Ora: " + p.getOraProiezione());  
    System.out.println("Biglietti: " + p.getNumeroBiglietti());  
    System.out.println("Costo unitario: " + p.getCostoUnitario());  
  
    double totale = p.getNumeroBiglietti() * p.getCostoUnitario();  
    System.out.println("Totale: " + totale);  
}
```

Elimina prenotazione

```
Scelta: 4  
Inserisci codice prenotazione:  
77ee60ce-fec0-41d7-8403-c0494da5a263  
Sei sicuro di voler eliminare la prenotazione? (s/n)  
s  
Prenotazione eliminata con successo  
Prenotazione eliminata con successo!  
  
Premere INVIO per continuare...
```

Questa operazione permette di cancellare un ordine esistente, liberando contestualmente i posti riservati all'interno della sala. La procedura si articola nelle seguenti fasi:

- **Inserimento codice:** il sistema richiede l'identificativo alfanumerico univoco della prenotazione che si desidera rimuovere.
- **Richiesta di conferma:** prima di procedere all'eliminazione definitiva, il programma mostra un messaggio di sicurezza (**Sei sicuro di voler eliminare la prenotazione? (s/n)**).
 - Digitando **s** (Si), l'operazione viene confermata e il sistema restituisce il messaggio **Prenotazione eliminata con successo**.

- Digitando **n** (No), l'operazione viene annullata in sicurezza senza apportare modifiche ai dati.

Il programma include un sistema di controllo per prevenire la cancellazione di dati errati. Qualora venga inserito un codice inesistente o digitato in modo errato, l'applicazione non subirà blocchi o arresti anomali.

In questo scenario, a seguito della conferma, il sistema restituirà semplicemente il messaggio di errore **Prenotazione non trovata**, lasciando inalterato l'archivio delle prenotazioni e consentendo all'utente di premere **Invio** per tornare al menu principale.

```
Scelta: 4
Inserisci codice prenotazione:
76trfghjhtrew
Sei sicuro di voler eliminare la prenotazione? (s/n)
s
Prenotazione non trovata
```

Controlli in Elimina Prenotazione

Prima di procedere all'eliminazione, il sistema esegue una serie di controlli finalizzati a verificare l'esistenza della prenotazione e la validità temporale dell'operazione.

- **Verifica dell'Esistenza della Prenotazione**

```
Prenotazione p = mappaPrenotazioni.get(codice);

if (p == null) {
    System.out.println("Prenotazione non trovata");
    return false;
}
```

Il metodo ricerca la prenotazione all'interno della struttura dati **mappaPrenotazioni** utilizzando il codice identificativo fornito.

- **Controllo Temporale**

```
// controllo data
if (p.getDataProiezione().isBefore(LocalDate.now()) || (p.getDataProiezione().equals(LocalDate.now()) && p.getOraProiezione().isBefore(LocalTime.now()))) {
    System.out.println("Impossibile eliminare: proiezione già avvenuta");
    return false;
}
```

Una volta individuata la prenotazione, il sistema verifica che la proiezione non sia già avvenuta.

Ripristino della Disponibilità della Sala

```
// recupero film
Film film = GestioneFilm.trovaProiezione(p.getDataProiezione(), p.getOraProiezione());

if (film != null) {
    film.aggiungiPosti(p.getNumeroBiglietti());
    GestioneFilm.salvaFilmSuFile();
}
```

Superati i controlli preliminari, il sistema individua la proiezione corrispondente mediante il metodo `GestioneFilm.trovaProiezione()`. Qualora la proiezione venga trovata, il numero di posti occupati dalla prenotazione viene restituito alla disponibilità della sala. Successivamente viene aggiornato il file contenente le informazioni delle proiezioni mediante il metodo `GestioneFilm.salvaFilmSuFile()`, mantenendo sincronizzati lo stato della sala e la disponibilità dei posti.

```
mappaPrenotazioni.remove(codice);

salvaPrenotazioniSuFile();

return true;
}
```

Una volta ripristinata la disponibilità della sala, la prenotazione viene rimossa dalla struttura dati `mappaPrenotazioni` mediante il metodo `remove()`. L'operazione comporta l'eliminazione definitiva della prenotazione dalla memoria dell'applicazione.

Visualizza proiezioni

La consultazione dei film in archivio sfrutta il *modulo centralizzato di scansione e filtraggio*. Per l'analisi dettagliata del flusso logico, della complessità computazionale e dei parametri del metodo, si rimanda al **Paragrafo 3. Palinsesto**.

Logout

```
GestioneUtenti.logout();
utenteLoggato = null;
System.out.println("Logout effettuato con successo.");
break;
```

La procedura di disconnessione permette di terminare in sicurezza la sessione di lavoro corrente, riportando l'applicazione allo stato iniziale di **Utente Guest**. A livello di backend, l'operazione non richiede la chiusura di connessioni persistenti, ma si basa sulla distruzione dei riferimenti agli oggetti utente precedentemente allocati in memoria.

Viene inizialmente invocato il metodo `logout()` che sovrascrive la variabile di stato interna alla classe di gestione con un riferimento null e restituisce un valore booleano per determinare l'esito dell'operazione.

```
public static boolean logout() {
    utenteCorrente = null;
    return true;
}
```

Ricevuto l'esito positivo, viene azzerato il riferimento locale tramite l'assegnazione `utenteLoggato = null;`. Questo passaggio è fondamentale affinché i cicli di controllo dei menu principali riconoscano la perdita dei privilegi di accesso e inibiscano la visualizzazione delle funzionalità avanzate riservate (Cliente, Proiezionista o Bigliettaio).

Il terminale stampa poi un messaggio di conferma e l'istruzione `break` consente al programma di reindirizzare il flusso verso il menu iniziale.

Chiudi programma

La selezione dell'opzione di uscita comporta l'interruzione immediata dell'applicazione. Il modulo delega la terminazione del processo alla routine centralizzata di chiusura della Java Virtual Machine, descritta in dettaglio nel **Paragrafo 4. Esci**

MENU PROIEZIONISTA

```
--- MENU PROIEZIONISTA ---  
1. Aggiungi proiezione  
2. Modifica proiezione  
3. Elimina proiezione  
4. Visualizza proiezioni  
5. Logout  
6. Chiudi programma  
  
Scelta:
```

Dopo aver effettuato l'accesso con le credenziali da Proiezionista, l'applicazione mostra il pannello di controllo dedicato alla gestione della programmazione delle sale. La navigazione segue il modello standard tramite input numerico e conferma con il tasto **Invio**.

Regola di integrità dei dati: per garantire la coerenza del sistema, non è consentito modificare o eliminare proiezioni per le quali è già stata effettuata una o più prenotazioni da parte dei Clienti.

All'interno di questa interfaccia sono disponibili le seguenti opzioni:

Aggiungi proiezione

```
Scelta: 1  
Titolo: Interstellar  
Genere: Sci-fi  
Regista: Christopher Nolan  
Anno: 2014  
Durata (minuti): 169  
Età minima: 8  
Data (AAAA-MM-GG): 2028-05-04  
Ora (HH:MM): 10:00  
Costo biglietto: 8  
Proiezione creata con successo!  
Film aggiunto con successo!  
  
Premere INVIO per continuare...
```

Consente di inserire un nuovo evento all'interno del palinsesto. La procedura guidata richiede l'immissione sequenziale di tutti i metadati del film: *Titolo*, *Genere*, *Regista*, *Anno di uscita*, *Durata (in minuti)*, *Età minima*, *Data (nel formato YYYY-MM-DD)*, *Ora (HH:mm)* e *Costo del biglietto*.

La funzione di inserimento di un nuovo evento in palinsesto è governata da una rigida logica di validazione. Prima di rendere persistente una proiezione, il sistema esegue una serie di controlli sequenziali per preservare l'integrità del database, sollevando l'eccezione personalizzata `FilmException` qualora un vincolo non venga soddisfatto.

Controlli Temporalì e di Validità dell'Input

Come evidenziato dai frammenti di codice, le prime verifiche riguardano la congruenza cronologica dei parametri inseriti rispetto all'istante di esecuzione:

- **Validazione della Data (Data non valida):** mediante l'utilizzo del metodo `data.isBefore(LocalDate.now())`, il sistema rigetta l'inserimento di eventi pianificati in date antecedenti a quella odierna.
- **Validazione dell'Orario (Orario non valido):** nel caso in cui la data coincida con quella corrente (`data.equals(LocalDate.now())`), il software verifica che l'orario d'inizio non sia già passato tramite la condizione `ora.isBefore(LocalTime.now())`.
- **Integrità dei Metadati:** viene accertato che l'età minima inserita non sia un valore negativo e che non esista un altro film avente gli stessi valori temporali di quello nuovo.

```
if(data.isBefore(LocalDate.now())) {  
    throw new FilmException("Data non valida");  
}else {  
    if(data.equals(LocalDate.now()) && ora.isBefore(LocalTime.now()))  
        throw new FilmException("Orario non valido");  
}
```

Algoritmo di Controllo delle Sovrapposizioni (controlloDurata)

L'aspetto più critico della logica di inserimento è affidato al metodo dedicato `controlloDurata()`. Questo metodo riceve come parametri la data, l'orario d'inizio, la durata del film e una chiave provvisoria identificativa.

L'algoritmo esegue un ciclo iterativo sulla mappa globale delle proiezioni esistenti (`mappaFile.values()`), isolando gli eventi programmati nella medesima giornata e nella medesima sala per calcolare i potenziali conflitti d'orario:

1. **Verifica Proiezione Precedente:** attraverso la classe `ChronoUnit.MINUTES.between()`, il sistema calcola la distanza temporale tra l'orario d'inizio dell'evento in esame e quello della nuova proiezione. Se la durata del film

già presente supera tale intervallo, l'inserimento viene interrotto poiché la visione precedente risulterebbe ancora in corso.

2. **Verifica Proiezione Successiva:** specularmente, se l'orario di inizio della nuova proiezione sommato alla sua durata invade lo slot orario dell'evento successivo in palinsesto, il sistema rileva l'incompatibilità dei tempi e restituisce un esito negativo.

In presenza di un conflitto, il software intercetta l'anomalia, stampa le specifiche della proiezione che ha generato la collisione oraria e blocca la transazione.

```
public static boolean controlloDurata(LocalDate data, LocalTime ora, int durata, String chiave) {
    for(Film f : mappaFilm.values()) {
        if(f.getData().equals(data) && !f.getChiave().equals(chiave)) {
            if(f.getOra().isBefore(ora)) {
                if(ChronoUnit.MINUTES.between(f.getOra(), ora)<f.getDurata()) {
                    System.out.println("Orario non disponibile, la seguente proiezione è ancora in corso "+f.toString());
                    return false;
                }
            } else {
                if(ChronoUnit.MINUTES.between(ora, f.getOra())<durata) {
                    System.out.println("Orario non disponibile, durata del film non compatibile con l'orario della proiezione successiva"+f.toString());
                    return false;
                }
            }
        }
    }
    return true;
}
```

Vincolo del Giorno Solare

Un ulteriore e fondamentale controllo logico è implementato per impedire che un film superi il limite della mezzanotte del giorno di proiezione. La classe `LocalTime` di Java interpreta nativamente lo scorrere del tempo all'interno del range diurno standard (00:00 - 23:59).

Se la somma dei minuti della durata porta l'orario di fine teorico a scavalcare la mezzanotte, l'espressione:

`ora.plusMinutes(durata).isBefore(ora)`

restituisce valore `true`. Questo avviene perché il nuovo orario calcolato "riparte" dall'inizio della giornata successiva, risultando temporalmente minore rispetto all'orario di inizio. Al verificarsi di questa condizione, il sistema solleva immediatamente un'eccezione `FilmException` con il messaggio "Il film non può protrarsi al giorno dopo".

```
if(ora.plusMinutes(durata).isBefore(ora)) {
    throw new FilmException("Il film non può protrarsi al giorno dopo");
}
```

Persistenza dei Dati (Caso di Successo)

Quando l'input supera l'intera catena di validazione, il sistema procede all'allocazione e alla scrittura permanente dei dati, secondo le modalità illustrate

```
String chiaveProvvisoria = "";
if(controlloDurata(data, ora, durata, chiaveProvvisoria)) {
    Film f = new Film(data, ora, titolo.trim(), genere.trim(), regista.trim(), anno, durata, etaMinima, costoBiglietto);
    mappaFilm.put(f.getChiave(), f);
    try(BufferedWriter bw = new BufferedWriter(new FileWriter(PATH, true))){
        String film = f.toCSV();
        bw.write(film);
        bw.newLine();
        return true;
    }
    catch(IOException e) {
        System.err.println("Errore nella creazione della proiezione" + e.getMessage());
    }
}
```

- **Allocazione in memoria:** viene istanziato un nuovo oggetto di tipo `Film` popolato con i parametri validati e allocato all'interno della struttura dati centralizzata `mappaFilm`.
- **Scrittura su File CSV:** il software apre un flusso di output stream gestito tramite un blocco *try-with-resources* che inizializza un `BufferedWriter` configurato in modalità *append* (`new FileWriter(PATH, true)`).
- **Serializzazione personalizzata:** l'oggetto viene serializzato in formato stringa CSV invocando il metodo `toCSV()`, il quale concatena i campi dell'oggetto intervallandoli con il delimitatore virgola (,). Il flusso provvede infine a scrivere la stringa sul file sul disco e ad aggiungere un terminatore di riga (`newLine()`).

Modifica proiezione

Permette di variare i parametri temporali di un evento già presente in palinsesto che non è ancora avvenuto. Per procedere, è necessario indicare i dettagli della proiezione da modificare e selezionare l'operazione desiderata dal sotto-menu numerato:

1. **Modifica data**
2. **Modifica ora**

Il metodo `modificaProiezione` accetta come parametri il film da modificare, un intero che determina il tipo di modifica (1 per modificare la data, 2 per modificare l'orario), e un parametro del tipo corrispondente alla scelta di modifica, determinato dall'uso di Varargs che consente di passare un numero indefinito di argomenti al metodo, permettendo di astrarre l'input indipendentemente dal fatto che si stia modificando una data o un orario.

```
public static boolean modificaProiezione(Film f, int tipoModifica, Object ...parametri) {
```

Il tipo di modifica viene determinato tramite uno *switch-case*, effettuando un controllo di validità del tipo del parametro tramite l'operatore booleano *instanceOf*, il quale istanzia e casta automaticamente le variabili in caso di esito positivo.

```
case 1:
    if(parametri[0] instanceof LocalDate localDate) {
```

```
case 2:
    if(parametri[0] instanceof LocalTime localTime) {
```

Come già detto, è possibile modificare una proiezione solo se priva di prenotazioni da parte dei clienti. Prima di invocare i metodi mutatori e rendere persistenti i dati, il software esegue un metodo preventivo: **Verifica dello stato delle prenotazioni**. Viene usato il metodo `getPostiSala()` riferito al film soggetto a modifica per accertare che il numero di posti liberi corrisponda esattamente alla capienza massima (sfruttando il campo statico e *final* della classe `Film`). La presenza anche di un solo biglietto venduto inibisce l'accesso al metodo, sollevando un'eccezione di sicurezza.

```
if (f.getPostiSala()==Film.capienza_max) {
```

Logica di Modifica e Prevenzione dei Conflitti di Identità

Il processo di modifica dei parametri temporali di una proiezione esistente eredita la medesima catena di validazione e i vincoli descritti per la fase di inserimento (controllo della data odierna, dell'orario antecedente e dello sfioramento del giorno solare).

Tuttavia, l'operazione di aggiornamento introduce una problematica specifica: la necessità di isolare l'entità sottoposta a modifica per evitare falsi positivi durante il controllo delle collisioni orarie. Per ovviare a questo problema il metodo `controlloDurata()` riceve come parametro aggiuntivo la stringa *chiave*, la quale identifica in modo univoco la proiezione che si intende aggiornare.

Durante il ciclo iterativo sulla mappa dei film, l'algoritmo applica la seguente condizione logica di filtro:

```
if(f.getData().equals(data) && !f.getChiave().equals(chiave)) {
```

Logica del Bypass: la presenza dell'operatore booleano di negazione (`!f.getChiave().equals(chiave)`) permette di escludere il film in fase di modifica dalle verifiche di sovrapposizione oraria.

Senza questa eccezione di identità, qualora un proiezionista decidesse di posticipare l'orario di un film (ad esempio, spostando l'inizio dalle 15:00 alle 15:30), l'algoritmo confronterebbe il nuovo input con il dato originale ancora memorizzato nell'archivio. Il sistema rileverebbe una sovrapposizione illecita con lo slot precedentemente occupato dall'evento stesso, generando un blocco anomalo dell'applicazione.

Viene inoltre verificato che l'evento non sia già avvenuto o in corso tramite il vincolo cronologico reale. In caso di violazione, il flusso si interrompe mostrando il messaggio **Impossibile modificare una proiezione già avvenuta**.

Caso di successo

Solo dopo il superamento dei controlli e l'esito positivo di `controlloDurata()`, lo stato dell'oggetto viene allineato in memoria eliminando la chiave originale e salvando nuovamente la proiezione nella `mappaFilm` con la nuova chiave identificativa (con data o orario modificati). Infine il film viene serializzato nuovamente sul file CSV sovrascrivendo il vecchio record.

Poiché la chiave identificativa dell'oggetto all'interno della `HashMap` (il campo calcolato tramite `f.getChiave()`) dipende strettamente dai parametri temporali della proiezione, una modifica alla data o all'orario altera l'indice stesso della mappa. Di conseguenza, il sistema provvede prima a rimuovere il vecchio record tramite `mappaFilm.remove(f.getChiave())`.

Viene invocato il rispettivo metodo setter (es. `f.setOra(localTime)`) per aggiornare lo stato interno dell'oggetto.

L'istanza modificata viene nuovamente allocata nella `HashMap` richiamando `mappaFilm.put(f.getChiave(), f)`. Questa operazione genera una nuova associazione chiave-valore basata sui parametri temporali aggiornati.

```
mappaFilm.remove(f.getChiave());
f.setOra(localTime);
mappaFilm.put(f.getChiave(), f);
salvaFilmSuFile();
System.out.println("Orario modificato!");
return true;
```

A differenza della fase di inserimento (dove i dati vengono aggiunti in modalità *append* in coda al file), l'aggiornamento di un record esistente richiede la riscrittura integrale dell'archivio per mantenere allineati i dati persistenti su disco.

Questa operazione è affidata al metodo `salvaFilmSuFile()`:

```

public static void salvaFilmSuFile() {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(PATH))){
        for(Film f : mappaFilm.values()) {
            bw.write(f.toCSV());
            bw.newLine();
        }
    }
    catch (IOException e) {
        System.err.println("Errore nel salvataggio dei film " + e.getMessage());
    }
}

```

Il metodo opera secondo la seguente logica:

- **Inizializzazione dello stream di output:** all'interno di un blocco *try-with-resources*, viene istanziato un `BufferedWriter` orientato verso il file CSV. In questo scenario, il `FileWriter` viene inizializzato **senza** il flag booleano di append (`new FileWriter(PATH)`), determinando la sovrascrittura controllata del file.
- **Iterazione e Serializzazione Globale:** il software esegue un ciclo *for-each* che scorre l'intera collezione aggiornata dei valori della mappa (`mappaFilm.values()`). Per ogni oggetto `Film` presente in memoria, viene invocato il metodo di serializzazione `.toCSV()`, scrivendo sequenzialmente le stringhe e separandole tramite un terminatore di riga (`bw.newLine()`).

In caso di anomalie fisiche di scrittura, il blocco *catch* intercetta un'eccezione di tipo `IOException`, stampando a terminale un messaggio di errore dedicato ("`Errore nel salvataggio dei film` ") senza provocare il crash dell'applicazione.

Elimina proiezione

Consente la rimozione di un evento dal palinsesto, a condizione che non vi siano prenotazioni attive collegate ad esso.

La rimozione definitiva di un evento dal palinsesto è gestita dal metodo `eliminaProiezione()`, il quale applica i medesimi criteri di validazione descritti per le fasi di inserimento e modifica.

Il processo si articola attraverso rigidi controlli sequenziali e la successiva riscrittura dell'archivio:

- **Validazione dei Vincoli di Sicurezza:** prima di procedere all'epurazione del record, l'algoritmo interroga lo stato della memoria per verificare due condizioni bloccanti:

1. *Presenza di prenotazioni:* il sistema controlla che il numero di posti liberi sia pari alla capienza massima della sala. Se è stato venduto anche un solo titolo d'ingresso, l'operazione fallisce per preservare l'integrità dei dati dei clienti.
 2. *Verifica cronologica reale:* viene accertato che la proiezione non sia antecedente all'istante corrente o in fase di svolgimento. In caso di violazione, il flusso viene interrotto sollevando un'eccezione che mostra a terminale il messaggio
Impossibile eliminare una proiezione già avvenuta.
- **Aggiornamento della Struttura Dati in Memoria:** una volta superata con successo la fase di validazione, il sistema provvede a disallocare l'oggetto. A differenza della modifica, l'eliminazione non richiede una nuova indicizzazione, ma si limita alla rimozione del record dalla mappa globale mediante l'invocazione del metodo nativo:

```
mappaFilm.remove(f.getChiave());  
salvaFilmSuFile();  
return true;
```

Questa operazione elimina definitivamente l'associazione e la chiave identificativa associata a quella specifica proiezione all'interno della **HashMap**. Per rendere permanente la cancellazione anche su disco, il flusso invoca il metodo di sintonizzazione globale **salvaFilmSuFile()**. Lo stream di output apre il file CSV in modalità di sovrascrittura controllata (**new FileWriter(PATH)**), ciclando sulla mappa aggiornata e riscrivendo da zero l'elenco dei soli film validi rimasti in memoria. L'evento rimosso viene così definitivamente escluso dal nuovo tracciato record generato sul disco.

Visualizza proiezioni

La consultazione dei film in archivio sfrutta il *modulo centralizzato di scansione e filtraggio*. Per l'analisi dettagliata del flusso logico, della complessità computazionale e dei parametri del metodo, si rimanda al **Paragrafo 3. Palinsesto**.

Logout

L'operazione coordina la disconnessione sicura dell'account dal terminale. Poiché la logica di azzeramento dei riferimenti in memoria (**utenteCorrente = null e utenteLoggato = null**) è centralizzata e speculare per ogni ruolo, si rimanda alla trattazione tecnica esaustiva e nella sezione **Menu Cliente >> Logout**.

Chiudi programma

La selezione dell'opzione di uscita comporta l'interruzione immediata dell'applicazione. Il modulo delega la terminazione del processo alla routine centralizzata di chiusura della Java Virtual Machine, descritta in dettaglio nel **Paragrafo 4. Esci**

MENU BIGLIETTAIO

```
--- MENU BIGLIETTAIO ---  
1. Visualizza prenotazioni di oggi  
2. Cerca prenotazione  
3. Logout  
4. Chiudi programma  
  
Scelta:
```

Gli utenti che effettuano l'autenticazione con il ruolo di Bigliettaio accedono a un pannello di controllo focalizzato sulla verifica e sulla ricerca delle prenotazioni a livello globale. La navigazione tra le opzioni segue la logica standard tramite input numerico.

All'interno di questo menu sono disponibili le seguenti funzionalità:

Visualizza prenotazioni di oggi

```
Scelta: 1  
===== PRENOTAZIONE =====  
Codice: e7a8b5a1-5ad4-4d12-b520-82420c3bece3  
Cliente: leog  
Data: 2026-06-29  
Ora: 10:30  
Biglietti: 2  
Costo unitario: 0.0  
Totale: 0.0  
  
Premere INVIO per continuare...
```

Questa opzione consente di monitorare in tempo reale tutti i titoli d'ingresso emessi per la data odierna. Per ciascuna prenotazione il sistema stampa a schermo:

- L'identificativo univoco (Codice).
- Lo username del Cliente che ha effettuato l'operazione.
- I parametri temporali della proiezione (Data odierna e Ora).
- La quantità di biglietti acquistati, il relativo costo unitario e l'importo totale della transazione.

Logica di Filtraggio

```
public static List<Prenotazione> prenotazioniOggi() {  
    List<Prenotazione> risultato = new ArrayList<>();  
    LocalDate oggi = LocalDate.now();  
    for (Prenotazione p : mappaPrenotazioni.values()) {  
        if (p.getDataProiezione().isEqual(oggi)) {  
            risultato.add(p);  
        }  
    }  
    return risultato;  
}
```

La selezione viene effettuata mediante un'iterazione completa della collezione delle prenotazioni. Per ogni elemento viene verificato il seguente vincolo:

- la data della proiezione deve essere uguale alla data corrente
(`p.getDataProiezione().isEqual(oggi)`)

Tutte le prenotazioni che soddisfano questa condizione vengono inserite nella lista dei risultati.

Cerca prenotazione

Questa funzionalità permette al Bigliettaio di eseguire ricerche mirate all'interno dell'intero archivio del sistema. L'applicazione consente di individuare una o più transazioni specifiche attraverso quattro diversi criteri di ricerca utilizzabili singolarmente:

```
Scelta: 2
Inserisci username: leog
===== PRENOTAZIONE =====
Codice: 1ac0ffdb-fdc1-4030-ae1c-a8bf7faf606c
Cliente: leog
Data: 2027-12-30
Ora: 10:30
Biglietti: 6
Costo unitario: 12.0
Totale: 72.0
===== PRENOTAZIONE =====
Codice: e7a8b5a1-5ad4-4d12-b520-82420c3bece3
Cliente: leog
Data: 2026-06-29
Ora: 10:30
Biglietti: 2
Costo unitario: 0.0
Totale: 0.0
===== PRENOTAZIONE =====
Codice: eff880a3-d2af-48a8-b95e-790a6d327bbb
Cliente: leog
Data: 2027-11-16
Ora: 10:30
Biglietti: 3
Costo unitario: 12.0
Totale: 36.0

Premere INVIO per continuare...
```

- **Codice prenotazione:** consente di risalire istantaneamente a una singola transazione inserendo l'identificativo alfanumerico univoco associato al biglietto.
- **Username del Cliente:** permette di monitorare l'attività di un utente specifico. Il sistema restituirà l'elenco sequenziale di tutte le prenotazioni collegate a quel profilo, indipendentemente dalla data della proiezione.
- **Titolo del film:** consente di visualizzare tutte le prenotazioni effettuate per una determinata opera cinematografica presente in palinsesto.
- **Intervallo di date:** permette di isolare le transazioni avvenute o programmate all'interno di un preciso arco temporale inserito.

Per ogni corrispondenza trovata, il programma stampa a schermo la scheda di riepilogo completa con tutti i dati della transazione: *Codice univoco*, *Username del cliente*, *Data e Ora della proiezione*, *Quantità di biglietti emessi*, *Costo unitario* e *Importo totale dovuto*. Al termine della consultazione, l'utente può premere il tasto **Invio** per ritornare al Menu Bigliettaio.

Modalità di ricerca

Il sistema supporta quattro differenti criteri di ricerca.

- **Ricerca per Codice**

```
public static Prenotazione cercaPerCodice(String codice) {  
  
    if (mappaPrenotazioni.containsKey(codice)) {  
        return mappaPrenotazioni.get(codice);  
    }  
  
    System.out.println("Prenotazione non trovata");  
    return null;  
}
```

La prenotazione viene individuata tramite il suo codice identificativo univoco. Il metodo verifica la presenza della chiave all'interno della struttura dati `mappaPrenotazioni`; se la ricerca ha esito positivo viene restituita la relativa prenotazione, altrimenti viene notificato il messaggio "Prenotazione non trovata".

- **Ricerca per Utente**

```
public static List<Prenotazione> cercaPerUtente(String username) {  
  
    List<Prenotazione> risultato = new ArrayList<>();  
  
    for (Prenotazione p : mappaPrenotazioni.values()) {  
  
        if (p.getUsername().equalsIgnoreCase(username)) {  
            risultato.add(p);  
        }  
    }  
  
    return risultato;  
}
```

Il sistema esegue una scansione completa della collezione delle prenotazioni confrontando lo username memorizzato con quello richiesto. Tutte le prenotazioni appartenenti allo stesso utente vengono inserite nella lista dei risultati.

- Ricerca per Titolo Film

```
public static List<Prenotazione> cercaPerFilm(String titolo) {  
    List<Prenotazione> risultato = new ArrayList<>();  
    for (Prenotazione p : mappaPrenotazioni.values()) {  
        if (p.getTitoloFilm().toLowerCase().startsWith(titolo.toLowerCase())) {  
            risultato.add(p);  
        }  
    }  
    return risultato;  
}
```

La ricerca viene effettuata confrontando il titolo della proiezione con la stringa inserita dall'utente. Il confronto è di tipo *case-insensitive* e utilizza il metodo `startsWith()`, consentendo di individuare anche prenotazioni il cui titolo inizia con la sequenza di caratteri specificata.

- Ricerca per Intervallo di Date

```
public static List<Prenotazione> cercaPerIntervalloDate(LocalDate inizio, LocalDate fine) {  
    List<Prenotazione> risultato = new ArrayList<>();  
    for (Prenotazione p : mappaPrenotazioni.values()) {  
        LocalDate data = p.getDataProiezione();  
        if ((data.isEqual(inizio) || data.isAfter(inizio)) && (data.isEqual(fine) || data.isBefore(fine))) {  
            risultato.add(p);  
        }  
    }  
    return risultato;  
}
```

Il sistema confronta la data della proiezione con due estremi temporali forniti in input. Vengono restituite tutte le prenotazioni comprese nell'intervallo, considerando entrambe le date come inclusive.

Dopodichè tramite uno switch si potrà selezionare una di queste ricerche:

```
public static List<Prenotazione> cercaPrenotazione(int tipoRicerca, String... parametri) {  
    switch (tipoRicerca) {  
        case 1:  
            // per codice  
            Prenotazione p = cercaPerCodice(parametri[0]);  
            return (p != null) ? List.of(p) : new ArrayList<>();  
        case 2:  
            // per utente  
            return cercaPerUtente(parametri[0]);  
        case 3:  
            // per titolo film  
            return cercaPerFilm(parametri[0]);  
        case 4:  
            // per intervallo date  
            LocalDate inizio = LocalDate.parse(parametri[0]);  
            LocalDate fine = LocalDate.parse(parametri[1]);  
            return cercaPerIntervalloDate(inizio, fine);  
        default:  
            System.out.println("Tipo ricerca non valido");  
            return new ArrayList<>();  
    }  
}
```

Logout

L'operazione coordina la disconnessione sicura dell'account dal terminale. Poiché la logica di azzeramento dei riferimenti in memoria (`utenteCorrente = null` e `utenteLoggato = null`) è centralizzata e speculare per ogni ruolo, si rimanda alla trattazione tecnica esaustiva e nella sezione **Menu Cliente >> Logout**.

Chiudi programma

La selezione dell'opzione di uscita comporta l'interruzione immediata dell'applicazione. Il modulo delega la terminazione del processo alla routine centralizzata di chiusura della Java Virtual Machine, descritta in dettaglio nel **Paragrafo 4. Esci**

2. Registrarsi

Questa funzione consente ai nuovi utenti di registrarsi autonomamente all'interno della piattaforma per ottenere le credenziali associate al ruolo di "Cliente". La procedura guidata richiede l'inserimento sequenziale dei dati anagrafici e di accesso dell'utente:

- **Dati Anagrafici:** vengono richiesti *Nome*, *Cognome* e *Domicilio*.
- **Data di Nascita:** deve essere inserita rispettando tassativamente il formato standard richiesto **YYYY-MM-DD** (Anno-Mese-Giorno).
- **Credenziali di Accesso:** vengono richiesti uno *Username* e una *Password*.

L'algoritmo di registrazione prevede due importanti meccanismi di controllo e sicurezza:

- **Crittografia delle Password:** per garantire la massima sicurezza e la riservatezza dei dati sensibili, il sistema applica una cifratura automatica sulla password prima di salvarla nel database locale. La chiave di accesso non viene mai memorizzata in chiaro, impedendone la visualizzazione a chiunque. La messa in sicurezza della chiave di accesso è affidata al metodo statico **hashPassword()**:

```
public static String hashPassword(String password) throws NoSuchAlgorithmException {  
    MessageDigest md = MessageDigest.getInstance("SHA-256");  
    byte[] hash = md.digest(password.getBytes());  
  
    // Converti i byte in una stringa esadecimale  
    StringBuilder sb = new StringBuilder();  
    for (byte b : hash)  
        sb.append(String.format("%02x", b)); //formato esadecimale a due caratteri  
  
    return sb.toString();  
}
```

Il metodo si serve della classe nativa di Java **MessageDigest** per inizializzare l'algoritmo di hashing **SHA-256** (*Secure Hash Algorithm*, a 256 bit), standard crittografico ampiamente riconosciuto per la sua robustezza contro attacchi e collisioni.

La stringa contenente la password in chiaro viene inizialmente convertita in un vettore di byte mediante il metodo **.getBytes()**.

Successivamente, l'istruzione **md.digest()** esegue l'elaborazione algoritmica, restituendo un array di byte (**byte[] hash**) che rappresenta l'impronta crittografica digitale, univoca e non invertibile, della password originaria.

Poiché l'array di byte risultante contiene caratteri non stampabili o non gestibili all'interno di un file CSV, il software esegue una routine di conversione in formato esadecimale:

Viene istanziato un oggetto **StringBuilder sb** e un ciclo *for-each* scorre sequenzialmente ogni singolo byte **b** dell'array crittografato, applicando la formattazione stringa: `sb.append(String.format("%02x", b));`

Al termine dell'iterazione, il metodo restituisce la stringa definitiva normalizzata a 64 caratteri (corrispondenti ai 256 bit dell'algoritmo), pronta per essere registrata in modo persistente nell'archivio o utilizzata come termine di confronto sicuro durante le procedure di login.

- **Verifica di disponibilità dello Username:** il sistema esegue un controllo di univocità in tempo reale sul database. Se l'identificativo scelto è già associato a un altro account (se la **mappaUtenti** contiene già la chiave identificativa determinata dallo username), il programma blocca l'operazione restituendo l'errore **Username non disponibile!**. In questo scenario, l'utente dovrà ripetere la procedura inserendo un identificativo differente.

```
if (mappaUtenti.containsKey(us)) {  
    System.out.print("Username non disponibile! ");  
    return false;  
}
```

In caso di esito positivo, viene creato un nuovo oggetto Cliente usando i parametri inseriti dall'utente in fase di registrazione:

```
Utente u = new Utente(n, c, us, hashedPassword, dn, d);  
mappaUtenti.put(u.getUsername(), u);  
try (BufferedWriter bw = new BufferedWriter(new FileWriter(PATH, true))) {  
    String utente = u.toCSV();  
    bw.write(utente);  
    bw.newLine();  
    System.out.println("Utente registrato correttamente! ");  
} catch (IOException e) {  
    System.err.println("Errore nella registrazione " + e.getMessage());  
    return false;  
}  
return true;  
}
```

Il nuovo cliente viene inserito nella HashMap corrispondente usando lo username come chiave. Successivamente il cliente viene inserito nel file CSV opportuno inizializzando un **BufferedWriter** in modalità *append* secondo il formato di scrittura previsto dal metodo `toCSV()`.

3. Palinsesto

Algoritmo Centralizzato di Scansione e Visualizzazione del Palinsesto

Il modulo dedicato alla consultazione e al filtraggio delle proiezioni costituisce una componente core del sistema, riutilizzata trasversalmente dall'applicazione. La funzione offre due modalità di esecuzione: l'estrazione integrale o il filtraggio incrementale basato su criteri combinati.

- **Estrazione Integrale del Palinsesto**

Quando l'utente nega l'applicazione dei filtri (digitando **n** al prompt iniziale), il sistema invoca il metodo `getTuttiFilm()`:

```
! --- PALINSESTO ATTUALE ---  
Vuoi applicare una ricerca? (s/n):
```

Il metodo istanzia e restituisce una nuova `ArrayList` popolata con la collezione dei valori della mappa globale (`mappaFilm.values()`). Questa scelta implementativa garantisce il disaccoppiamento dei dati, evitando che eventuali manipolazioni della lista dei risultati interferiscano con l'integrità della `HashMap` originaria:

```
public static List<Film> getTuttiFilm() {  
    return new ArrayList<>(mappaFilm.values());  
}
```

- **Logica del Filtraggio Progressivo (`retainAll`)**

Se l'utente richiede una ricerca avanzata (digitando **s**), il backend inizializza una lista di lavoro denominata `risultati` (popolata inizialmente tramite `getTuttiFilm()`) e vi applica in cascata i filtri attivati.

Il software adotta il metodo `.retainAll()` per eseguire l'intersezione logica tra gli insiemi: ogni filtro genera una sottolista provvisoria di corrispondenze e `retainAll()` provvede a eliminare dalla lista principale tutte le istanze che non rispettano il nuovo criterio (usando un AND logico).

- *Filtro per titolo:* supporta la ricerca parziale e *case-insensitive*. Può essere saltato premendo direttamente **Invio**.

```

if (titolo != null && !titolo.isBlank()) {
    risultati.retainAll(
        trovaPerTitolo(titolo)
    );
}

```

Previa verifica che la stringa inserita non sia nulla o vuota (`!titolo.isBlank()`), viene invocato il metodo ausiliario `trovaPerTitolo(titolo)`, il quale istanzia una lista d'appoggio e scorre la mappa dei film tramite un ciclo *for-each*. La condizione di corrispondenza normalizza i caratteri (usando il `toLowerCase`) e verifica l'uguaglianza della porzione iniziale della stringa passata per parametro (usando `startsWith`). I film validi vengono salvati nella lista d'appoggio e passati al metodo d'intersezione.

```

public static LinkedList<Film> trovaPerTitolo(String t){
    LinkedList<Film> l = new LinkedList<>();
    for (Film f : mappaFilm.values()) {
        if((f.getTitolo().toLowerCase()).startsWith(t.toLowerCase()))
            l.add(f);
    }
    return l;
}

```

- *Filtro per Genere*: segue la medesima logica del titolo, supportando la ricerca parziale e *case-insensitive*. Può essere saltato premendo direttamente **Invio**.
- *Filtro per Date*: permette di differenziare la ricerca temporale attraverso un sotto-menu numerato. Le opzioni disponibili includono:
 - **0**: nessun filtro applicato.
 - **1**: ricerca dei film programmati dopo una determinata data (inclusa).
 - **2**: ricerca dei film programmati prima di una determinata data (inclusa).
 - **3**: ricerca dei film inseriti all'interno di un preciso intervallo di date (incluse).

Il programma riconosce la scelta tramite uno *switch-case* e, in base al filtro scelto, applica il criterio di ricerca usando la classe enumerativa interna `GestioneFilm.Criterio`.

```

switch (sceltaData) {
    case 1 -> critData =
        GestioneFilm.Criterio.DOPO_DI;

    case 2 -> critData =
        GestioneFilm.Criterio.PRIMA_DI;

    case 3 -> {
        critData =
            GestioneFilm.Criterio.COMPRESO_TRA;

        System.out.print(
            "Seconda data: "
        );

        data2 =
            LocalDate.parse(
                sc.nextLine()
            );
    }
}

```

Prima di accedere alla struttura dati dei film il sistema esegue un controllo di validità dei dati inseriti: se il criterio corrisponde ad un intervallo di date e la seconda viene omessa, viene sollevata un'eccezione personalizzata `FormatoDatiNonValidoException`, interrompendo il flusso.

```

if (criterioDate != null && data1 != null) {
    if (criterioDate == Criterio.COMPRESO_TRA && data2 == null)
        throw new FormatoDatiNonValidoException(
            "Serve una seconda data"
        );

    risultati.retainAll(
        trovaPerDate(
            criterioDate,
            data1,
            data2
        )
    );
}

```

Superati i controlli viene invocato il metodo ausiliario `trovaPerDate`, che esegue uno `switch` sul `Criterio` e cicla sulla `mappaFilm` effettuando i confronti cronologici della classe `LocalDate`:

```

public static LinkedList<Film> trovaPerDate(Criterio crit, LocalDate data1, LocalDate data2){
    LinkedList<Film> l = new LinkedList<>();
    switch(crit) {
        case COMPRESO_TRA:
            if(data2.isBefore(data1)) throw new FormatoDatiNonValidoException("Date non valide");
            for (Film f : mappaFilm.values()) {
                if((f.getData().equals(data1) || data1.isBefore(f.getData())) && (f.getData().equals(data2) || data2.isAfter(f.getData())))
                    l.add(f);
            }
            return l;

            //nei casi di PRIMA_DI e DOPO_DI la seconda data è messa a null
        case PRIMA_DI:
            for (Film f : mappaFilm.values()) {
                if (f.getData().isBefore(data1))
                    l.add(f);
            }
            return l;

        case DOPO_DI:
            for (Film f : mappaFilm.values()) {
                if(f.getData().isAfter(data1))
                    l.add(f);
            }
            return l;

        default:
            System.out.println("Ricerca non valida");
            return l;
    }
}

```

case COMPRESO_TRA: verifica che la data d'inizio non sia posteriore alla data di fine e isola i film la cui data coincide con gli estremi o si colloca internamente al range indicato.

case PRIMA_DI e **case DOPO_DI:** utilizzano rispettivamente i metodi booleani `.isBefore(data1)` e `.isAfter(data1)` impostando a `null` il secondo parametro non richiesto.

- **Filtro Costo:** funziona in modo analogo al filtro data, sostituendo i metodi cronologici con i relativi operatori di confronto numerico per isolare le tariffe.

I film risultanti dalla ricerca vengono salvati in una lista finale (`filmList`), che viene stampata a terminale tramite un ciclo *for-each* che chiama il metodo `visualizzaProiezione()`, il quale stampa i dettagli di un film in modo da renderli chiari all'utente mostrando tutti i dettagli.

```

for (Film f : filmList) {

    System.out.println(
        "\n-----"
    );

    GestioneFilm.visualizzaProiezione(f);
}

```

```

public static void visualizzaProiezione(Film f) {
    System.out.println("==== PROIEZIONE =====");
    System.out.println("Titolo: " + f.getTitolo());
    System.out.println("Data e ora: " + f.getData().toString() + " " + f.getOra().toString());
    System.out.println("Genere: " + f.getGenere());
    System.out.println("Regista e anno di uscita: " + f.getRegista() + " " + f.getAnno());
    System.out.println("Durata ed età minima: " + f.getDurata() + " minuti, " + f.getEtaMinima() + " anni");
    System.out.println("Costo del biglietto: " + f.getCostoBiglietto() + " euro");
    System.out.println("Posti liberi: " + f.getPostiSala());
}

```

4. Esci

La funzione di chiusura controllata dell'applicazione, invocabile dalle diverse schermate principali tramite l'opzione dedicata, garantisce l'interruzione pulita di tutti i moduli attivi.

```

System.out.println("Arrivederci!");
System.exit(0);
break;

```

La procedura esegue una sequenza di terminazione strutturata a livello di sistema operativo:

- **Messaggio di congedo:** il sistema provvede innanzitutto a stampare a terminale una stringa di saluto standardizzata ("Arrivederci!") per confermare visivamente all'utente la presa in carico del comando.
- **Arresto controllato della JVM (`System.exit`):** l'effettiva chiusura del programma è affidata all'invocazione del metodo statico nativo `System.exit(0)`. Questo metodo interrompe l'esecuzione della Java Virtual Machine (JVM) corrente.
 - *Significato dello Status Code 0:* il passaggio del parametro intero 0 rappresenta la convenzione standard nei sistemi operativi per indicare una **terminazione normale e priva di errori** (*successful termination*). Questo segnala al sistema ospite che l'applicazione ha concluso il proprio ciclo di vita in modo atteso, senza sollevare eccezioni o riscontrare crash di runtime.
- **Rottura del Flusso di Controllo (`break`):** l'istruzione finale `break`; serve a interrompere formalmente la struttura condizionale (il costrutto `switch-case` del menu) all'interno della quale è inserita l'opzione di uscita. Sebbene a runtime diventi di fatto irraggiungibile a causa del precedente arresto immediato della JVM tramite `System.exit(0)`, la sua inclusione preserva la corretta sintassi del blocco di controllo e previene eventuali warning in fase di compilazione.

DATA SET

Logica di Singularità delle Strutture Dati e Classi Statiche

Le classi deputate alla logica di business e alla persistenza (es. `GestioneFilm`, `GestioneUtenti`): **adottano esclusivamente membri e metodi statici e costruttori privati.**

Questa scelta architetturale risponde a una precisa necessità di *integrità dei dati*:

- **Prevenzione delle istanze multiple:** rendendo le classi interamente statiche, si inibisce la possibilità di istanziare molteplici oggetti di tipo "Gestione" all'interno del ciclo di vita dell'applicazione.
- **Consistenza delle HashMap e dei File:** se le classi di gestione fossero state istanziabili, l'allocazione di più oggetti avrebbe potuto generare duplicazioni locali delle `HashMap` deputate alla memoria di lavoro. Di conseguenza, modifiche concorrenti o asincrone su istanze diverse avrebbero compromesso l'allineamento dei dati, provocando corruzioni irreversibili o perdite di informazioni durante la sovrascrittura dei rispettivi file CSV su disco.

La scelta della staticità garantisce quindi l'esistenza di **un unico punto di accesso globale** e centralizzato per ogni entità, assicurando che tutte le operazioni di inserimento, modifica ed eliminazione eseguite dai diversi menu (Cliente, Proiezionista, Bigliettaio) insistano sempre sulle medesime strutture dati in memoria.

Credenziali di Accesso per i Ruoli Utente

Per agevolare la fase di verifica e correzione del progetto, l'applicazione viene fornita con un **Data Set di Test** già pre-caricato all'interno dei file di archivio della cartella principale (es. `data`). Questo permette di testare immediatamente tutte le funzionalità del software senza dover popolare manualmente il database da zero.

Di seguito vengono elencati i dati di esempio e gli account di test pre-configurati per ciascun livello di accesso:

Proiezionisti

- **Mario Rossi**
 - Username: `mariorossi`
 - Password: `rossi10`
- **Riccardo Volontè**
 - Username: `ricky1`
 - Password: `r67`

Bigliettai

- **Ilaria Esposito**
 - Username: **ila-ila**
 - Password: **rob02**
- **Federica Sabatini**
 - Username: **fedez**
 - Password: **fs20**
- **Luca Zanzi**
 - Username: **iznazacul**
 - Password: **zaNzu**
- **Matilde Bralia**
 - Username: **matss**
 - Password: **MB77**
- **Lorenzo Malena**
 - Username: **malens9**
 - Password: **Din@**

Clienti

- **Leonardo Giuliani**
 - Username: **leog**
 - Password: *(Impostata tramite registrazione)*
- **Paolo Andrioli**
 - Username: **trexrobot55**
 - Password: *(Impostata tramite registrazione)*
- **Mike Ross**
 - Username: **mikk**
 - Password: *(Impostata tramite registrazione)*

Archivio Proiezioni e Storico Prenotazioni

Il database contiene già una serie di dati storici e simulazioni ideali per testare i sistemi di ricerca e di controllo del programma:

- **Palinsesto Film:** sono già presenti diverse proiezioni configurate a sistema con date, orari, sale e costi differenti. Questo consente di testare subito l'efficacia dei filtri combinati (ricerca per titolo, genere, range di costo e intervallo di date).
- **Storico Prenotazioni:** nel sistema sono già caricate alcune prenotazioni. Queste transazioni sono fondamentali per verificare:
 1. La visualizzazione e la modifica lato **Cliente**.

2. La ricerca globale per username o per codice univoco (UUID) lato **Bigliettaio**.
3. Il blocco di sicurezza lato **Proiezionista**, che impedirà la modifica o l'eliminazione dei film che hanno già questi biglietti associati.

LIMITI

Durante l'utilizzo quotidiano del programma, è importante tenere a mente alcune limitazioni strutturali dell'interfaccia, pensate per evitare errori di compilazione e garantire la sicurezza dei dati:

- **Compilazione dei moduli e annullamento:** quando si avvia una procedura guidata (come l'aggiunta di una nuova proiezione o l'inserimento di una prenotazione), il sistema richiede di inserire tutte le informazioni una dopo l'altra. **Non è possibile annullare l'operazione a metà strada.** Inoltre, l'applicazione verificherà la correttezza dei dati (es. se l'orario è valido o se ci sono posti liberi) solo alla fine dell'intera compilazione. Se viene commesso un errore all'inizio, la notifica comparirà soltanto dopo aver premuto Invio sull'ultimo dato richiesto.
- **Cancellazione degli account:** una volta che un profilo utente viene registrato all'interno del sistema, **l'account non può essere eliminato.** Questa scelta è necessaria per preservare lo storico delle prenotazioni e dei biglietti emessi all'interno del cinema, che rimarranno associati in modo permanente al rispettivo nome utente.
- **Ricerca delle prenotazioni (Lato Bigliettaio):** all'interno del menu riservato al Bigliettaio, la ricerca delle prenotazioni dei clienti può essere effettuata **esclusivamente tramite lo Username** dell'utente. Il sistema non supporta la ricerca inserendo il Nome e il Cognome anagrafici del cliente. Si consiglia pertanto di tenere sempre a portata di mano lo username esatto dell'utente per poter gestire o verificare i suoi biglietti.